

Nama : Yudhistira Putra Hartanto

Kelas/Absen : 1G/29

NIM : 254107020083

Praktikum Implementasi Binary Search Tree menggunakan Linked List (100 Menit)

Percobaan 1

- Class Mahasiswa29.java
package Pertemuan14;

```
public class Mahasiswa29 {
    String nim;
    String nama;
    String kelas;
    double ipk;

    public Mahasiswa29(){

    }

    public Mahasiswa29(String nim, String nama, String kelas, double ipk){
        this.nim = nim;
        this.nama = nama;
        this.kelas = kelas;
        this.ipk = ipk;
    }

    public void tampilInformasi(){
        System.out.println("NIM : " + this.nim + " "+
        "NAMA : " + this.nama + " "+
        "KELAS : " + this.kelas + " "+
        "IPK : " + this.ipk);
    }
}
```

- Class Node29.java
package Pertemuan14;

```
public class Node29 {
    Mahasiswa29 mahasiswa;
    Node29 left, right;

    public Node29(){

    }

    public Node29 (Mahasiswa29 mahasiswa){
```

```

        this.mahasiswa = mahasiswa;
        left = right = null;
    }
}

```

- Class BinaryTree29.java

```
package Pertemuan14;
```

```

public class BinaryTree29 {
    Node29 root;

    public BinaryTree29(){
        root = null;
    }

    public boolean isEmpty(){
        return root == null;
    }

    public void add(Mahasiswa29 mahasiswa) {
        Node29 newNode = new Node29(mahasiswa);
        if (isEmpty()) {
            root = newNode;
        } else {
            Node29 current = root;
            Node29 parent = null;
            while (true) {
                parent = current;
                if (mahasiswa.ipk < current.mahasiswa.ipk) {
                    current = current.left;
                    if (current == null) {
                        parent.left = newNode;
                        return;
                    }
                } else {
                    current = current.right;
                    if (current == null) {
                        parent.right = newNode;
                        return;
                    }
                }
            }
        }
    }

    boolean find(double ipk) {
        boolean result = false;
        Node29 current = root;
        while (current != null) {
            if (current.mahasiswa.ipk == ipk) {
                result = true;
                break;
            } else if (ipk > current.mahasiswa.ipk) {

```

```

        current = current.right;
    } else {
        current = current.left;
    }
}
return result;
}

void traversePreOrder(Node29 node) {
    if (node != null) {
        node.mahasiswa.tampilInformasi();
        traversePreOrder(node.left);
        traversePreOrder(node.right);
    }
}

void traverseInOrder(Node29 node) {
    if (node != null) {
        traverseInOrder(node.left);
        node.mahasiswa.tampilInformasi();
        traverseInOrder(node.right);
    }
}

void traversePostOrder(Node29 node) {
    if (node != null) {
        traversePostOrder(node.left);
        traversePostOrder(node.right);
        node.mahasiswa.tampilInformasi();
    }
}

Node29 getSuccessor(Node29 del) {
    Node29 successor = del.right;
    Node29 successorParent = del;
    while (successor.left != null) {
        successorParent = successor;
        successor = successor.left;
    }
    if (successor != del.right) {
        successorParent.left = successor.right;
        successor.right = del.right;
    }
    return successor;
}

void delete(double ipk) {
    if (isEmpty()) {
        System.out.println("Binary tree kosong");
        return;
    }

    // cari node (current) yang akan dihapus

```

```

Node29 parent = root;
Node29 current = root;
boolean isLeftChild = false;

while (current != null) {
    if (current.mahasiswa.ipk == ipk) {
        break;
    } else if (ipk < current.mahasiswa.ipk) {
        parent = current;
        current = current.left;
        isLeftChild = true;
    } else if (ipk > current.mahasiswa.ipk) {
        parent = current;
        current = current.right;
        isLeftChild = false;
    }
}

// penghapusan
if (current == null) {
    System.out.println("Data tidak ditemukan");
    return;
} else {
    // jika tidak ada anak (leaf), maka node dihapus
    if (current.left == null && current.right == null) {
        if (current == root) {
            root = null;
        } else {
            if (isLeftChild) {
                parent.left = null;
            } else {
                parent.right = null;
            }
        }
    }
    } else if (current.left == null) { // jika hanya punya 1 anak (kanan)
        if (current == root) {
            root = current.right;
        } else {
            if (isLeftChild) {
                parent.left = current.right;
            } else {
                parent.right = current.right;
            }
        }
    }
    } else if (current.right == null) { // jika hanya punya 1 anak (kiri)
        if (current == root) {
            root = current.left;
        } else {
            if (isLeftChild) {
                parent.left = current.left;
            } else {
                parent.right = current.left;
            }
        }
    }
}

```

```

    }
} else { // jika punya 2 anak
    Node29 successor = getSuccessor(current);
    System.out.println("Jika 2 anak, current = ");
    successor.mahasiswa.tampilInformasi();
    if (current == root) {
        root = successor;
    } else {
        if (isLeftChild) {
            parent.left = successor;
        } else {
            parent.right = successor;
        }
    }
    successor.left = current.left;
}
}
}
}
}

```

- Class BinaryTreeMain29.java
package Pertemuan14;

```

public class BinaryTreeMain29 {
    public static void main(String[] args) {
        BinaryTree29 bst = new BinaryTree29();

        bst.add(new Mahasiswa29("244160121", "Ali", "A", 3.57));
        bst.add(new Mahasiswa29("244160221", "Badar", "B", 3.85));
        bst.add(new Mahasiswa29("244160185", "Candra", "C", 3.21));
        bst.add(new Mahasiswa29("244160220", "Dewi", "B", 3.54));

        System.out.println("\nDaftar semua mahasiswa (in order traversal):");
        bst.traverseInOrder(bst.root);

        System.out.println("\nPencarian data mahasiswa:");
        System.out.print("Cari mahasiswa dengan ipk: 3.54 : ");
        String hasilCari = bst.find(3.54) ? "Ditemukan" : "Tidak ditemukan";
        System.out.println(hasilCari);

        System.out.print("Cari mahasiswa dengan ipk: 3.22 : ");
        hasilCari = bst.find(3.22) ? "Ditemukan" : "Tidak ditemukan";
        System.out.println(hasilCari);

        bst.add(new Mahasiswa29("244160131", "Devi", "A", 3.72));
        bst.add(new Mahasiswa29("244160205", "Ehsan", "D", 3.37));
        bst.add(new Mahasiswa29("244160170", "Fizi", "B", 3.46));
        System.out.println("\nDaftar semua mahasiswa setelah penambahan 3
mahasiswa:");
        System.out.println("\nOrder Traversal:");
        bst.traverseInOrder(bst.root);
        System.out.println("\nPreOrder Traversal:");
    }
}

```

```

        bst.traversePreOrder(bst.root);
        System.out.println("\nPostOrder Traversal:");
        bst.traversePostOrder(bst.root);

        System.out.println("\nPenghapusan data mahasiswa");
        bst.delete(3.57);
        System.out.println("\nDaftar semua mahasiswa setelah penghapusan 1
mahasiswa (in order traversal):");
        bst.traverseInOrder(bst.root);
    }
}

```

- Hasil

Daftar semua mahasiswa (in order traversal):

NIM : 244160185 NAMA : Candra KELAS : C IPK : 3.21

NIM : 244160220 NAMA : Dewi KELAS : B IPK : 3.54

NIM : 244160121 NAMA : Ali KELAS : A IPK : 3.57

NIM : 244160221 NAMA : Badar KELAS : B IPK : 3.85

Pencarian data mahasiswa:

Cari mahasiswa dengan ipk: 3.54 : Ditemukan

Cari mahasiswa dengan ipk: 3.22 : Tidak ditemukan

Daftar semua mahasiswa setelah penambahan 3 mahasiswa:

Order Traversal:

NIM : 244160185 NAMA : Candra KELAS : C IPK : 3.21

NIM : 244160205 NAMA : Ehsan KELAS : D IPK : 3.37

NIM : 244160170 NAMA : Fizi KELAS : B IPK : 3.46

NIM : 244160220 NAMA : Dewi KELAS : B IPK : 3.54

NIM : 244160121 NAMA : Ali KELAS : A IPK : 3.57

NIM : 244160131 NAMA : Devi KELAS : A IPK : 3.72

NIM : 244160221 NAMA : Badar KELAS : B IPK : 3.85

PreOrder Traversal:

NIM : 244160121 NAMA : Ali KELAS : A IPK : 3.57

NIM : 244160185 NAMA : Candra KELAS : C IPK : 3.21

NIM : 244160220 NAMA : Dewi KELAS : B IPK : 3.54

NIM : 244160205 NAMA : Ehsan KELAS : D IPK : 3.37

NIM : 244160170 NAMA : Fizi KELAS : B IPK : 3.46

NIM : 244160221 NAMA : Badar KELAS : B IPK : 3.85

NIM : 244160131 NAMA : Devi KELAS : A IPK : 3.72

PostOrder Traversal:

NIM : 244160170 NAMA : Fizi KELAS : B IPK : 3.46

NIM : 244160205 NAMA : Ehsan KELAS : D IPK : 3.37

NIM : 244160220 NAMA : Dewi KELAS : B IPK : 3.54

NIM : 244160185 NAMA : Candra KELAS : C IPK : 3.21

NIM : 244160131 NAMA : Devi KELAS : A IPK : 3.72

NIM : 244160221 NAMA : Badar KELAS : B IPK : 3.85

NIM : 244160121 NAMA : Ali KELAS : A IPK : 3.57

Penghapusan data mahasiswa

Jika 2 anak, current =

NIM : 244160131 NAMA : Devi KELAS : A IPK : 3.72

Daftar semua mahasiswa setelah penghapusan 1 mahasiswa (in order traversal):

NIM : 244160185 NAMA : Candra KELAS : C IPK : 3.21

NIM : 244160205 NAMA : Ehsan KELAS : D IPK : 3.37

NIM : 244160170 NAMA : Fizi KELAS : B IPK : 3.46

NIM : 244160220 NAMA : Dewi KELAS : B IPK : 3.54

NIM : 244160131 NAMA : Devi KELAS : A IPK : 3.72

NIM : 244160221 NAMA : Badar KELAS : B IPK : 3.85

- **Pertanyaan**

1. Mengapa dalam binary search tree proses pencarian data bisa lebih efektif dilakukan dibanding binary tree biasa?
2. Untuk apakah di class Node, kegunaan dari atribut left dan right?
3. a. Untuk apakah kegunaan dari atribut root di dalam class BinaryTree?
b. Ketika objek tree pertama kali dibuat, apakah nilai dari root?
4. Ketika tree masih kosong, dan akan ditambahkan sebuah node baru, proses apa yang akan terjadi?
5. Perhatikan method add(), di dalamnya terdapat baris program seperti di bawah ini. Jelaskan secara detil untuk apa baris program tersebut?
6. Jelaskan langkah-langkah pada method delete() saat menghapus sebuah node yang memiliki dua anak. Bagaimana method getSuccessor() membantu dalam proses ini?

Jawaban

1. Karena binary search tree memiliki aturan nilai kecil di kiri dan nilai besar di kanan, sehingga pencarian bisa langsung memilih jalur kiri atau kanan tanpa harus menelusuri semua node.
2. left menunjuk ke anak kiri (nilai lebih kecil), right menunjuk ke anak kanan (nilai lebih besar).
3. a. root menunjuk ke node paling atas/pertama dalam pohon yang menjadi titik awal semua operasi.
b. null
4. Node baru langsung dijadikan sebagai root tanpa proses perbandingan karena pohon kosong.
5. Kode tersebut berfungsi untuk menentukan posisi node baru. Caranya dengan membandingkan IPK, lalu bergerak ke kiri jika lebih kecil atau ke kanan jika lebih besar. Saat menemukan posisi kosong (null), node baru ditempatkan di situ.
6. Langkah-langkahnya adalah diawali dengan mencari successor (node paling kiri dari subtree kanan), lalu ganti node yang dihapus dengan successor, dan pindahkan subtree kiri ke successor, getSuccessor() membantu mencari node pengganti yang tepat agar struktur BST tetap terjaga.

Percobaan 2

- **Class BinaryTreeArray29**
package Pertemuan14;

```
public class BinaryTreeArray29 {  
    Mahasiswa29[] dataMahasiswa;
```

```

int idxLast;

public BinaryTreeArray29() {
    this.dataMahasiswa = new Mahasiswa29[10];
}

void populateData(Mahasiswa29 dataMhs[], int idxLast) {
    this.dataMahasiswa = dataMhs;
    this.idxLast = idxLast;
}

void traverseInOrder(int idxStart) {
    if (idxStart <= idxLast) {
        if (dataMahasiswa[idxStart] != null) {
            traverseInOrder(2 * idxStart + 1);
            dataMahasiswa[idxStart].tampilInformasi();
            traverseInOrder(2 * idxStart + 2);
        }
    }
}
}

```

- Class BinaryTreeArrayMain29

```
package Pertemuan14;
```

```

public class BinaryTreeArrayMain29 {
    public static void main(String[] args) {
        BinaryTreeArray29 bta = new BinaryTreeArray29();

        Mahasiswa29 mhs1 = new Mahasiswa29("244160121", "Ali", "A", 3.57);
        Mahasiswa29 mhs2 = new Mahasiswa29("244160185", "Candra", "C", 3.41);
        Mahasiswa29 mhs3 = new Mahasiswa29("244160221", "Badar", "B", 3.75);
        Mahasiswa29 mhs4 = new Mahasiswa29("244160220", "Dewi", "B", 3.35);
        Mahasiswa29 mhs5 = new Mahasiswa29("244160131", "Devi", "A", 3.48);
        Mahasiswa29 mhs6 = new Mahasiswa29("244160205", "Ehsan", "D", 3.61);
        Mahasiswa29 mhs7 = new Mahasiswa29("244160170", "Fizi", "B", 3.86);

        Mahasiswa29[] dataMahasiswa = {mhs1, mhs2, mhs3, mhs4, mhs5, mhs6, mhs7,
        null, null, null};
        int idxLast = 6;
        bta.populateData(dataMahasiswa, idxLast);

        System.out.println("\nInorder Traversal Mahasiswa: ");
        bta.traverseInOrder(0);
    }
}

```

- Hasil

Inorder Traversal Mahasiswa:

NIM : 244160220 NAMA : Dewi KELAS : B IPK : 3.35
 NIM : 244160185 NAMA : Candra KELAS : C IPK : 3.41
 NIM : 244160131 NAMA : Devi KELAS : A IPK : 3.48
 NIM : 244160121 NAMA : Ali KELAS : A IPK : 3.57

NIM : 244160205 NAMA : Ehsan KELAS : D IPK : 3.61

NIM : 244160221 NAMA : Badar KELAS : B IPK : 3.75

NIM : 244160170 NAMA : Fizi KELAS : B IPK : 3.86

- **Pertanyaan**

1. Apakah kegunaan dari atribut data dan idxLast yang ada di class BinaryTreeArray?
2. Apakah kegunaan dari method populateData()?
3. Apakah kegunaan dari method traverseInOrder()?
4. Jika suatu node binary tree disimpan dalam array indeks 2, maka di indeks berapakah posisi left child dan right child masing-masing?
5. Apa kegunaan statement `int idxLast = 6` pada praktikum 2 percobaan nomor 4?
6. Mengapa indeks $2 \times \text{idxStart} + 1$ dan $2 \times \text{idxStart} + 2$ digunakan dalam pemanggilan rekursif, dan apa kaitannya dengan struktur pohon biner yang disusun dalam array?

Jawaban

1. dataMahasiswa menyimpan array node pohon, idxLast menunjukkan indeks terakhir yang berisi data.
2. Kegunaannya adalah mengisi array dataMahasiswa dan menentukan nilai idxLast dari luar kelas.
3. Kegunaannya adalah menampilkan semua data mahasiswa secara **in order** (kiri -> node -> kanan) dengan rekursif menggunakan indeks array.
4. Left child di indeks 5 ($2 \times 2 + 1$), right child di indeks 6 ($2 \times 2 + 2$).
5. Menunjukkan bahwa data terakhir berada di indeks ke-6 (artinya ada 7 data, indeks 0 sampai 6 yang valid).
6. Rumus tersebut merepresentasikan hubungan orangtua-anak dalam representasi complete binary tree menggunakan array, di mana anak kiri = $2 \times \text{indeks} + 1$ dan anak kanan = $2 \times \text{indeks} + 2$.